

5.8 - Rollback: critérios, automação e o custo de decisões irreversíveis

Voltar é estratégia, não derrota

Voltar, quando é a decisão certa

Rollback como ferramenta de operação, não como sinal de fracasso

- **Rollback restaura a versão anterior conhecida como estável**
 - Decisão tomada quando degradação detectada não é trivialmente corrigível
- **Time-to-recover importa mais que diagnóstico completo durante incidente**
 - Primeiro restaurar serviço, depois investigar a causa raiz com calma
- **Rollback não é fracasso, é maturidade operacional**
 - Times maduros têm tempo médio de rollback abaixo de cinco minutos
- **Quanto mais rápido o rollback, mais agressivos os deploys podem ser**
 - Confiança em voltar é o que permite avançar com velocidade

Critérios objetivos, SLOs e error budgets

Quando voltar deixa de ser opinião e vira regra

- **SLO define o limite aceitável de uma métrica de qualidade do serviço**
 - Disponibilidade, latência percentil 99, taxa de erro por minuto
- **Error budget é a tolerância restante antes do SLO ser violado**
 - Mensal, semanal ou em janela contínua conforme criticidade
- **Rollback automatizado dispara quando degradação consome budget rápido demais**
 - Detecção em poucos minutos, decisão em segundos, execução em segundos
- **Critérios são acordados em tempo calmo, não no meio do incidente**
 - Discussão em incidente vira opinião, decisão automatizada vira processo

A migration que não podia voltar

- **Migration adiciona coluna `delivery_type` como NOT NULL sem valor padrão**
 - Deploy sobe normalmente, serviço inicializa, mas inserções começam a falhar

LUNALOG

Um deploy aparentemente trivial da LunaLog incluía uma migration que adicionava a coluna `delivery_type` como NOT NULL, sem valor padrão. O serviço subiu, a aplicação inicializou, mas 20% das inserções começaram a falhar com `constraint violation`, registros antigos que tentavam ser atualizados não tinham o novo campo. Reverter a aplicação levou cinco minutos. Reverter a migration sem perder os dados já inseridos pela nova versão exigiu três horas de trabalho cuidadoso, com risco real de perda. O time descobriu naquela tarde que rollback de aplicação é trivial, rollback de schema é cirurgia. O padrão expand-contract teria tornado essa migration completamente reversível.

Expand-contract, a coreografia reversível

Schema evolui em passos atômicos, cada um reversível

- **Expand, adicionar novo schema sem remover o antigo, compatível com versão atual**
 - Coluna nova adicionada como nullable, valor padrão definido, sem constraint nova
- **Migrate, aplicação dual-write, escreve no formato antigo e no novo simultaneamente**
 - Backfill dos dados antigos preenche o novo formato em batches controlados
- **Contract, com toda aplicação usando o novo formato, o antigo é removido**
 - Migration de remoção feita semanas ou meses depois, com toda a leitura migrada
- **Cada passo é deployável independente, cada passo é reversível**
 - Custo é coreografia mais longa, ganho é rollback sempre possível

Decisões que tornam o rollback impossível

- **Algumas mudanças são tecnicamente irreversíveis em produção real**

Migrations destrutivas de coluna, mudanças de tipo sem fallback, exclusão de mensagens em filas, alteração de chaves de partição em bancos distribuídos, tudo isso são pontos sem retorno. Decisões assim precisam ser tratadas como projeto, com comunicação, janela e plano de contingência. Tratá-las como deploy normal é trocar conforto agora por risco de catástrofe depois.

- Lista de mudanças irreversíveis precisa estar explícita na documentação do time

Fechamento da seção 5, o que ficou estabelecido

- ✓ Versionamento semântico comunica impacto, mudança quebrada comunicada é melhor que silenciosa
- ✓ Consumer-driven contracts deslocam responsabilidade do contrato para quem consome
- ▶ CI, Continuous Delivery e Continuous Deployment são três níveis com pré-requisitos distintos
- ✓ Pipeline maduro tem stages ordenadas, gates calibrados e artefatos auditáveis
- ▶ Blue/green, rolling, canary e feature flags atendem riscos diferentes
- ! Rollback rápido viabiliza velocidade, irreversibilidade exige tratamento especial



Sistema escalável, seguro, com pipeline maduro, deploys controlados e rollback planejado. E então acontece um incidente às 3h da manhã. O serviço está 'up' segundo o dashboard. A CPU está normal. A memória está normal. Mas algo está claramente errado e você não consegue descobrir o quê. A próxima seção resolve esse problema, observabilidade não é monitoramento, é a capacidade de fazer perguntas que você ainda não sabe que vai precisar fazer.

Seção 6, Observabilidade e Monitoramento